

Apollo Institute of Computer Application

BCA Semester – 2

Assignment – Advanced Web Technology

Unit 2 : Working with Arrays, Forms and Functions

Submission Date : 13/03/2026

Using Arrays

✓7 MARKS QUESTIONS

1. Define an array. Explain its types (Indexed, Associative, Multidimensional) with example.

Definition of Array

An **array** in PHP is a special variable that is used to store **multiple values in a single variable**. Instead of creating separate variables for each value, we store all values inside one array.

❑ **Example without array:**

```
$sub1 = "Maths";
```

```
$sub2 = "English";
```

```
$sub3 = "Science";
```

❑ **Example with array:**

```
$subjects = array("Maths", "English", "Science");
```

Types of Arrays in PHP

PHP mainly supports **three types of arrays**:

1. Indexed Array
 2. Associative Array
 3. Multidimensional Array
-

1. Indexed Array

An indexed array stores values with **numeric index numbers** (starting from 0).

✓Example:

```
<?php
$colors = array("Red", "Green", "Blue");
echo $colors[0]; // Output: Red
echo $colors[1]; // Output: Green
?>
```

□ Here:

- Red → index 0
 - Green → index 1
 - Blue → index 2
-

2. Associative Array

An associative array stores values using **key-value pairs** instead of numeric index.

✓Example:

```
<?php
$student = array(
    "name" => "Payal",
    "roll" => 12,
    "course" => "BCA"
);
echo $student["name"]; // Output: Payal
echo $student["course"]; // Output: BCA
?>
```

□ Here:

- "name", "roll", "course" are keys
 - "Payal", 12, "BCA" are values
-

3. Multidimensional Array

A multidimensional array is an array that contains **one or more arrays inside another array**.

It is mainly used to store data in **table format (rows and columns)**.

✓ Example:

```
<?php
$students = array(
    array("Payal", 12, "BCA"),
    array("Riya", 15, "BBA"),
    array("Amit", 20, "BCom")
);
echo $students[0][0]; // Output: Payal
echo $students[1][2]; // Output: BBA
?>
```

□ Explanation:

- First index → row
- Second index → column

2. What is a Multidimensional Array? Explain with suitable example.

Definition

A **Multidimensional Array** is an array that contains **one or more arrays inside another array**.

In simple words, it is an **array of arrays**.

It is mainly used to store data in **tabular form (rows and columns)** like student records, marksheets, product lists, etc.

Syntax

```
$arrayName = array(
    array(value1, value2, value3),
    array(value1, value2, value3)
);
```

Example : Simple Multidimensional Array

```
<?php
$students = array(
    array("Payal", 12, "BCA"),
    array("Riya", 15, "BBA"),
    array("Amit", 20, "BCom")
);
// Accessing elements
echo $students[0][0]; // Output: Payal
echo $students[1][2]; // Output: BBA
?>
```

Explanation:

First index → represents **row**

Second index → represents **column**

So,

`students[0][0]` → First row, first column

`$students[1][2]` → Second row, third column

3. Explain array sorting functions in PHP with example.

1. sort()

Sorts an **indexed array in ascending order**

Re-indexes the array (index starts from 0 again)

✓Example:

```
<?php
$numbers = array(40, 10, 30, 20);
sort($numbers);
foreach($numbers as $value){
    echo $value . " ";
}
```

```
}
```

```
// Output: 10 20 30 40
```

```
?>
```

2. **rsort()**

Sorts an indexed array in **descending order**

✓**Example:**

```
<?php
```

```
$numbers = array(40, 10, 30, 20);
```

```
rsort($numbers);
```

```
foreach($numbers as $value){
```

```
    echo $value . " ";
```

```
}
```

```
// Output: 40 30 20 10
```

```
?>
```

3. **asort()**

Sorts an **associative array by value** in ascending order

Maintains key-value relationship

✓**Example:**

```
<?php
```

```
$marks = array("Payal"=>80, "Riya"=>70, "Amit"=>90);
```

```
asort($marks);
```

```
foreach($marks as $key => $value){
```

```
    echo $key . " : " . $value . "<br>";
```

```
}
```

```
?>
```

□ Output (sorted by value):

Riya : 70

Payal : 80

Amit : 90

4. arsort()

Sorts associative array by value in **descending order**

✔Example:

```
<?php
$marks = array("Payal"=>80, "Riya"=>70, "Amit"=>90);
arsort($marks);
foreach($marks as $key => $value){
    echo $key . " : " . $value . "<br>";
}
?>
```

5. ksort()

Sorts associative array by **key** in ascending order

✔Example:

```
<?php
$marks = array("Payal"=>80, "Riya"=>70, "Amit"=>90);
ksort($marks);
foreach($marks as $key => $value){
    echo $key . " : " . $value . "<br>";
}
?>
```

6. krsort()

Sorts associative array by **key in descending order**

✔Example:

```
<?php
$marks = array("Payal"=>80, "Riya"=>70, "Amit"=>90);
krsort($marks);
```

```
foreach($marks as $key => $value){  
    echo $key . " : " . $value . "<br>";  
}  
?>
```

4. Explain string to array and array to string conversion in PHP (explode() and implode()).

string to Array and Array to String Conversion in PHP

In PHP, we can convert:

String → **Array** using `explode()`

Array → **String** using `implode()`

These functions are very useful when handling user input, CSV data, tags, etc.

1. explode() Function (String → Array)

Definition:

The `explode()` function splits a string into an array using a specified separator (delimiter).

Syntax:

`explode(separator, string);`

separator → Character used to split the string (like comma, space, etc.)

string → The original string

✓Example:

```
<?php  
$str = "Apple,Banana,Mango";  
$fruits = explode(",", $str);  
print_r($fruits);  
?>
```

Output:

Array

```
(  
    [0] => Apple  
    [1] => Banana  
    [2] => Mango  
)
```

□ **Explanation:**

The comma (,) is used as separator.

The string is divided into array elements.

2. implode() Function (Array → String)

Definition:

The `implode()` function joins array elements into a single string using a specified separator.

Syntax:

`implode(separator, array);`

separator → Character inserted between elements

array → The array to join

✓ **Example:**

```
<?php  
$colors = array("Red", "Green", "Blue");  
$str = implode(" ", $colors);  
echo $str;  
?>
```

Output:

Red, Green, Blue

□ **Explanation:**

", " is used as separator.

Array elements are combined into one string.

Difference Between explode() and implode()

Function	Conversion Type	Purpose
explode()	String → Array	Splits string into array
implode()	Array → String	Joins array into string

5. Write a PHP program to create an array from form input and display its elements.

Below is a simple example where the user enters values in a form, and PHP stores them in an array and displays the elements.

✔ Step 1: HTML Form + PHP Code

```
<?php
if(isset($_POST['submit']))
{
    // Creating array from form input
    $items = array(
        $_POST['item1'],
        $_POST['item2'],
        $_POST['item3']
    );
    echo "<h3>Array Elements are:</h3>";
    // Displaying array elements
    foreach($items as $value)
    {
        echo $value . "<br>";
    }
}
?>

<!DOCTYPE html>

<html>
```

```
<head>
  <title>Array from Form</title>
</head>
<body>
<h2>Enter Three Items</h2>
<form method="post">
  Item 1: <input type="text" name="item1"><br><br>
  Item 2: <input type="text" name="item2"><br><br>
  Item 3: <input type="text" name="item3"><br><br>
  <input type="submit" name="submit" value="Submit">
</form>
</body>
</html>
```

Explanation

- The user enters three values in the form.
 - When the form is submitted, `$_POST` collects the input values.
 - These values are stored inside an array `$items`.
 - `foreach()` loop is used to display each element of the array.
-

✓ 1 MARK QUESTIONS

1. What is an array?

An array is a variable that stores multiple values in a single variable.

2. Define indexed array.

An indexed array stores elements with numeric index numbers starting from 0.

3. Define associative array.

An associative array stores elements using key-value pairs.

4. Define multidimensional array.

A multidimensional array is an array that contains one or more arrays inside it.

5. Write syntax to create an array in PHP.

```
$array = array(value1, value2, value3);
```

OR

```
$array = [value1, value2, value3];
```

6. How to add a new element to an array?

```
$array[] = "NewValue";
```

7. Which loop is mainly used to access array elements?

foreach loop.

8. Write syntax of sort() function.

```
sort($array_name);
```

9. Write syntax of explode() function.

```
explode(separator, string);
```

10. Write syntax of implode() function.

```
implode(separator, array);
```

11. Which function is used to count elements of an array?

count () function.

12. Which function is used to search an element in an array?

in_array () function.

13. Which function is used to merge two arrays?

`array_merge()` function.

14. What is the use of `array_push()`?

It is used to add one or more elements at the end of an array.

15. What is the use of `array_pop()`?

It removes the last element from an array.

Creating Web Application

✓ 7 MARKS QUESTIONS

1. Differentiate between `include()` and `require()` with example.

Both `include()` and `require()` functions are used to insert the content of one PHP file into another PHP file.

They help in code reusability (like header, footer, database connection file, etc.).

1. `include()`

Generates a **warning** if the file is not found.

Script **continues execution** even after error.

Used when file is **not mandatory**.

✓ **Example:**

```
<?php
```

```
include("header.php"); // If file not found, warning is shown
```

```
echo "Welcome to my website!";
```

```
?>
```

□ If `header.php` is missing:

Warning will appear.

"Welcome to my website!" will still be printed.

2. require()

Generates a **fatal error** if the file is not found.

Script **stops execution** immediately.

Used when file is **essential/mandatory**.

✓Example:

```
<?php
require("config.php"); // If file not found, fatal error occurs
echo "Connected to database!";
?>
```

□ If `config.php` is missing:

Fatal error will occur.

"Connected to database!" will NOT be printed.

2. Explain form handling using GET and POST methods in PHP.

Form handling in PHP means collecting user input from an HTML form and processing it using PHP.

PHP mainly uses two methods to send form data:

- **GET method**
 - **POST method**
-

1. Form Handling Using GET Method

Definition:

The **GET method** sends form data through the URL.

The data is visible in the browser's address bar.

Characteristics:

- Data is visible in URL
- Limited data size
- Less secure
- Used for search forms or non-sensitive data

✔Example (GET Method)

```
<!DOCTYPE html>

<html>

<body>

<form method="get">

    Name: <input type="text" name="name">

    <input type="submit" value="Submit">

</form>

<?php
if(isset($_GET['name']))
{
    $name = $_GET['name'];
    echo "Welcome " . $name;
}
?>

</body>

</html>
```

□ URL Example:

<http://localhost/form.php?name=Payal>

2. Form Handling Using POST Method

Definition:

The **POST method** sends form data through the HTTP request body.
The data is **not visible** in the URL.

Characteristics:

- Data is not shown in URL
- Can send large amount of data
- More secure than GET
- Used for login forms, registration, passwords

✓ Example (POST Method)

```
<!DOCTYPE html>

<html>

<body>

<form method="post">

    Name: <input type="text" name="name">

    <input type="submit" value="Submit">

</form>

<?php
if(isset($_POST['name']))
{
    $name = $_POST['name'];
    echo "Welcome " . $name;
}
?>

</body>

</html>
```

Difference Between GET and POST

Feature	GET	POST
Data Visibility	Visible in URL	Not visible
Data Size	Limited	Large amount
Security	Less secure	More secure
Usage	Search, filters	Login, forms

3. What is a sticky form? Explain with suitable example.

Definition

A **Sticky Form** is a form that **retains (keeps) previously entered values** after the form is submitted.

It means when the user submits the form and there is an error, the entered data is **not lost**. The form fields remain filled with the previous values.

Sticky forms improve **user experience**, especially in validation.

Why We Use Sticky Forms?

- To avoid re-entering data
- To show validation errors
- To improve usability

✓Example of Sticky Form

```
<?php
$name = "";
$email = "";
if($_SERVER["REQUEST_METHOD"] == "POST")
{
    $name = $_POST["name"];
    $email = $_POST["email"];
    if(empty($name) || empty($email))
    {
        echo "All fields are required!<br>";
    }
    else
    {
        echo "Form submitted successfully!";
    }
}
```

```
?>
<!DOCTYPE html>
<html>
<body>
<h2>Sticky Form Example</h2>
<form method="post">
Name:
<input type="text" name="name"
value="<?php echo htmlspecialchars($name); ?>">
<br><br>
Email:
<input type="text" name="email"
value="<?php echo htmlspecialchars($email); ?>">
<br><br>
<input type="submit" value="Submit">
</form>
</body>
</html>
```

✔ Output Behavior

If user enters:

- Name: Payal
- Email: (empty)

It will show:

All fields are required!

But the **Name field will still contain "Payal"**.

4. Explain the header() function in PHP. How is it used for redirection?

Definition

The `header()` function in PHP is used to **send raw HTTP headers** to the browser.

It is mainly used for:

- Page redirection
- Setting content type
- Preventing caching
- Forcing file download

Important Rule

❑ The `header()` function must be called **before any output** (like echo or HTML).

Otherwise, it will give an error:

```
Headers already sent
```

Syntax

```
header("header_string");
```

Using header() for Redirection

To redirect a user from one page to another, we use:

```
header("Location: filename.php");
```

```
exit();
```

`exit()` ; is used to stop further script execution.

✔ Example: Simple Redirection

```
<?php
```

```
header("Location: welcome.php");
```

```
exit();
```

```
?>
```

❑ When this file runs, it automatically redirects to **welcome.php**.

✔ Example: Redirection After Form Submission

```
<?php
```

```
if(isset($_POST['submit']))
{
    $username = $_POST['username'];
    if($username == "Payal")
    {
        header("Location: success.php");
        exit();
    }
    else
    {
        header("Location: error.php");
        exit();
    }
}
?>
<form method="post">
    Username: <input type="text" name="username">
    <input type="submit" name="submit" value="Login">
</form>
```

5. Explain Date and Time handling in PHP with important functions.

PHP provides built-in functions to create, format, and manipulate date and time.

These functions help in displaying current date, calculating time differences, storing timestamps, etc.

1. date() Function

Definition:

The `date()` function is used to **format and display the current date and time**.

Syntax:

date(format, timestamp);

- **format** → Specifies output format
- **timestamp** → Optional (current time if not given)

✓Example:

```
<?php
echo date("d-m-Y"); // Output: 02-03-2026
echo "<br>";
echo date("h:i:s"); // Current time
?>
```

□ Common Format Characters:

Format	Meaning
d	Day (01–31)
m	Month (01–12)
Y	4-digit year
h	Hour (12-hour)
H	Hour (24-hour)
i	Minutes
s	Seconds

2. time() Function

Definition:

The `time()` function returns the **current timestamp** (number of seconds since Jan 1, 1970).

✓Example:

```
<?php
echo time();
?>
```

□ Timestamp is useful for calculations and storing date in database.

3. mktime() Function

Definition:

`mktime()` creates a timestamp for a specific date and time.

✓Syntax:

```
mktime(hour, minute, second, month, day, year);
```

✓**Example:**

```
<?php
$timestamp = mktime(10, 30, 0, 3, 2, 2026);
echo date("d-m-Y h:i:s", $timestamp);
?>
```

4. strtotime() Function

✓**Definition:**

strtotime() converts a text date string into a timestamp.

✓**Example:**

```
<?php
echo date("d-m-Y", strtotime("next Monday"));
?>
```

☐ Very useful for date calculations.

5. date_default_timezone_set()

✓**Definition:**

Sets the default timezone.

✓**Example:**

```
<?php
date_default_timezone_set("Asia/Kolkata");
echo date("h:i:s");
?>
```

6. getdate() Function

✓**Definition:**

Returns an associative array containing date information.

✓**Example:**

```
<?php
```

```
print_r(getdate());
```

```
?>
```

6. Explain how external files are used in PHP applications.

Definition

External files in PHP are separate files (like .php, .html, .txt) that are included inside another PHP file using functions like:

- `include()`
- `require()`
- `include_once()`
- `require_once()`

They help in **code reusability, modular programming, and easy maintenance.**

Why We Use External Files?

- To reuse common code (header, footer, menu)
 - To separate database connection
 - To make code organized and readable
 - To reduce duplication
-

Functions Used to Include External Files

1 ☐ `include()`

- Shows warning if file not found
- Script continues execution

```
<?php
```

```
include("header.php");
```

```
echo "Welcome to my website!";
```

```
?>
```

2 ☐ `require()`

- Shows fatal error if file not found

- Script stops execution

```
<?php
require("config.php");
echo "Connected successfully!";
?>
```

3 ☐ **include_once()**

- Includes file only once
- Prevents duplicate inclusion

```
<?php
include_once("menu.php");
?>
```

4 ☐ **require_once()**

- Includes file once
- Stops execution if file missing

```
<?php
require_once("dbconnect.php");
?>
```

Example of Using External Files

✓ **header.php**

```
<h1>My Website Header</h1>
```

✓ **footer.php**

```
<footer>Copyright 2026</footer>
```

✓ **index.php**

```
<?php
include("header.php");

?>

<p>Welcome to homepage.</p>

<?php
include("footer.php");
```

?>

✓ 1 MARK QUESTIONS

1. Define template in PHP.

A template in PHP is a predefined layout or structure used to separate design from logic.

2. What is an external file?

An external file is a separate file included in a PHP program using include() or require().

3. Write syntax of include() function.

```
include("filename.php");
```

4. Write syntax of require() function.

```
require("filename.php");
```

5. What is a constant?

A constant is a fixed value that cannot be changed during script execution.

6. Write syntax of define() function.

```
define("CONSTANT_NAME", value);
```

7. Write syntax of date() function.

```
date(format, timestamp);
```

8. What is sticky form?

A sticky form retains previously entered values after form submission.

9. Name two superglobal variables used in form handling.

`$_GET` and `$_POST`

10. Write syntax of mail() function.

`mail(to, subject, message, headers);`

11. What is output buffering?

Output buffering stores output in memory before sending it to the browser.

12. Write syntax of ob_start() function.

`ob_start();`

13. What is HTTP header?

An HTTP header is information sent to the browser before actual output.

14. Write syntax of header() function.

`header("header_information");`

15. Which function is used to redirect a page in PHP?

`header()` function.

Creating Functions

✓ 7 MARKS QUESTIONS

1. Define a function in PHP. Explain how to create and use a simple function with example.

Definition

A **function** in PHP is a block of reusable code that performs a specific task.

Instead of writing the same code again and again, we can create a function and call it whenever needed.

Functions help in:

- Reducing code repetition
 - Improving readability
 - Making program modular
-

❑ How to Create a Function

✔Syntax:

```
function function_name()
{
    // code to be executed
}
```

- `function` → Keyword used to declare function
- `function_name` → Name of the function
- `{ }` → Block of code

How to Call (Use) a Function

After defining a function, we call it using its name followed by parentheses:

```
function_name();
```

Example: Simple Function

```
<?php
// Creating a function
function greet()
{
    echo "Hello, Welcome to PHP!";
}
// Calling the function
greet();
?>
```

✔Output:

Hello, Welcome to PHP!

Explanation

1. `function greet()` → Creates the function
2. Code inside `{ }` executes when function is called

3. `greet () ;` → Calls the function

2. Explain functions with arguments in PHP. How are arguments passed to a function? Give example.

Definition

A **function with arguments** is a function that accepts input values when it is called. These input values are called **arguments (parameters)**.

Arguments allow functions to work with **different values** without changing the function code.

Syntax

```
function function_name($parameter1, $parameter2)
{
    // code using parameters
}

function_name(value1, value2); // Passing arguments
```

Example: Function with Two Arguments

```
<?php
function add($a, $b)
{
    $sum = $a + $b;
    echo "Sum = " . $sum;
}

add(10, 20);

?>
```

✔Output:

Sum = 30

```
<?php
```

```

function test($x)
{
    $x = $x + 5;

    echo "Inside Function: " . $x . "<br>";
}

$num = 10;
test($num);
echo "Outside Function: " . $num;

?>

```

Output:

Inside Function: 15

Outside Function: 10

☐ Original value remains unchanged.

2☐ Pass by Reference

- The actual variable is passed.
- Changes inside function affect original variable.
- Use & symbol.

✓Example:

```

<?php
function test(&$x)
{
    $x = $x + 5;
}

$num = 10;
test($num);
echo "After Function Call: " . $num;

?>

```

Output:

After Function Call: 15

- ❑ Original value is changed.

3. Explain variable scope in PHP (Local, Global, Static) with example.

Definition

Variable scope refers to the **area of a program where a variable can be accessed**.

In PHP, there are mainly **three types of variable scope**:

1. Local Scope
2. Global Scope
3. Static Scope

1. Local Scope

Definition:

A variable declared **inside a function** has local scope.

It can be used **only within that function**.

✔Example:

```
<?php
function test()
{
    $x = 10; // Local variable
    echo "Inside Function: " . $x;
}
test();
// echo $x; // Error (Cannot access local variable outside function)
?>
```

❑ Explanation:

- \$x is created inside the function.
- It cannot be accessed outside the function.

2. Global Scope

Definition:

A variable declared **outside a function** has global scope.

It cannot be accessed directly inside a function unless we use the `global` keyword.

✔Example:

```
<?php
$x = 20; // Global variable

function test()
{
    global $x; // Access global variable
    echo "Inside Function: " . $x;
}

test();

?>
```

□ Explanation:

- `$x` is declared outside the function.
- To use it inside the function, we use `global $x;`.

3. Static Scope

Definition:

Normally, when a function finishes execution, its local variables are deleted.

But if we declare a variable as `static`, it **retains its value** even after the function ends.

✔Example:

```
<?php

function counter()
{
    static $count = 0;
```

```

$count++;

echo $count . "<br>";
}

counter();

counter();

counter();

?>

```

✓Output:

```

1
2
3

```

□ Explanation:

- `$count` is static.
- It remembers its previous value after each function call.

□ Difference Between Local, Global and Static

Scope	Declared	Accessed	Lifetime
Local	Inside function	Only inside function	Until function ends
Global	Outside function	Everywhere (using global keyword)	Whole program
Static	Inside function	Only inside function	Retains value between calls

3. Differentiate between local variable and global variable in PHP.

Difference Between Local Variable and Global Variable in PHP (Table Format)

Basis of Difference	Local Variable	Global Variable
Definition	A variable declared inside a function.	A variable declared outside a function.
Place of Declaration	Inside function body.	Outside all functions.
Scope / Accessibility	Accessible only within that function.	Accessible throughout the program.
Access Inside	Directly accessible inside same	Must use <code>global</code> keyword or <code>\$GLOBALS</code> array

Basis of Difference	Local Variable	Global Variable
Function	function only.	to access inside function.
Lifetime	Exists only during function execution.	Exists during the entire script execution.
Example Declaration	function test() { \$x = 10; }	\$x = 20; (outside function)

4.Explain the complete structure of creating and calling a function in PHP.

Definition

A **function** in PHP is a reusable block of code that performs a specific task. It is created using the `function` keyword and executed by calling its name.

1. Complete Structure of Function

```
<?php
// Step 1: Function Declaration (Creation)
function function_name($parameter1, $parameter2)
{
    // Step 2: Function Body (Statements)
    // Code to be executed
    return $value; // (Optional) Return statement
}
// Step 3: Function Call
function_name(argument1, argument2);
?>
```

Explanation of Structure

Part	Description
<code>function</code>	Keyword used to declare a function
<code>function_name</code>	Name of the function
Parameters	Variables inside parentheses (optional)

Part	Description
Function Body	Code inside { }
return	Sends value back to calling statement (optional)
Function Call	Executes the function using its name

✓ 1 MARK QUESTIONS

1. 1. What is a function in PHP?

A function is a reusable block of code that performs a specific task.

2. 2. Write syntax for creating a function.

```
function function_name()
{
    // code
}
```

3. 3. How do you call a function in PHP?

By writing the function name followed by parentheses:

```
function_name();
```

4. 4. What is an argument in a function?

An argument is a value passed to a function when it is called.

5. 5. What is a default argument value?

A default argument value is a predefined value used if no argument is passed.

6. 6. Which keyword is used to return a value from a function?

return keyword.

7. What is variable scope?

Variable scope defines where a variable can be accessed in a program.

8. Define local variable.

A local variable is declared inside a function and can be used only within that function.

9. Define global variable.

A global variable is declared outside a function and can be accessed throughout the script.

10. Which keyword is used to access global variable inside a function?

`global` keyword.